WAVER

Master Product & Architecture Baseline v1.1

MASTER PRODUCT & ARCHITECTURE BASELINE

WAVER

Enterprise AI Agent Platform

Documento mestre consolidado: visão do produto, Agent Builder, domínios, marketplace privado, revendas, contratos, entitlements, operação comercial e arquitetura obrigatória de microsserviços.

North Star

Uma plataforma B2B2B para criar, integrar, executar e governar agentes de IA. A WAVER fornece a plataforma e o catálogo; revendas comercializam e administram clientes; clientes utilizam somente os recursos contratados. A complexidade técnica fica abaixo de uma experiência de produto clara.

Plano	Responsabilidade
Commercial Plane	Revendas, clientes, catálogo, marketplace, contratos, entitlements, billing e licenciamento.
Agent Plane	Agent Builder, Objective Engine, Flow Engine, Runtime, Memory e Evaluation.
Capability Plane	Tools, connectors, knowledge e channels.
Automation Plane	Triggers, workflows, scheduler e eventos.
Platform Plane	Identity, tenancy, audit, observability, notifications e usage.

Versão 1.0 • Outubro de 2026

1. Escopo do produto

WAVER não é apenas um SDR, CRM ou chatbot. É uma plataforma de agentes empresariais, capaz de suportar atendimento, tarefas e automações sobre sistemas existentes, com aplicações nativas opcionais.

- Agent Builder como núcleo de autoria, mas não como único produto.
- Integração com CRMs, ERPs, canais, APIs e sistemas externos em vez de tentar substituí-los.
- Biblioteca/Marketplace privado de Tools e capabilities reutilizáveis.

- Aplicações nativas opcionais, como CRM, Support Inbox e Scheduling, construídas sobre os mesmos contratos da plataforma.
- Arquitetura multi-tenant, multi-domain, versionada, observável e governável.

1.1 Princípios de produto

Princípio	Regra
Simple surface, powerful underneath	A interface abstrai detalhes técnicos sem remover capacidade.
Prompt abstraction	Usuário configura objetivos, fluxos, políticas e comportamento; prompt técnico é compilado internamente.
Domain isolation	Comercial, Suporte, Marketing etc. são bounded contexts organizacionais.
Composable capabilities	Agents consomem Tools/Connectors por contratos claros.
Governed autonomy	Autonomia limitada por policies, permissions, approvals, quotas e audit.
Version everything important	Agent definitions, flows, tools, policies e deployments possuem versões.

2. Modelo organizacional e domínios

Organization → Workspace → Domain → Agent → AgentVersion → Deployment

Elemento	Função
Organization	WAVER, Revenda ou Cliente como entidade organizacional.
Workspace	Ambiente operacional da organização cliente.
Domain	Fronteira de negócio: Comercial, Suporte, Marketing, Financeiro ou custom.
Agent	Unidade inteligente pertencente a um Domain.
AgentVersion	Snapshot imutável/versionado da configuração.
Deployment	Versão publicada em um ambiente.

- Cada Domain possui Agents, Knowledge, dados e políticas próprios.

-

Um agente não deve compartilhar dados entre Domains implicitamente.

-

Identidade e autorização respeitam Organization, Workspace, Domain e recurso.

3. Agent Builder

O Builder define agentes; o Runtime executa versões publicadas. A experiência principal não é uma caixa de system prompt.

3.1 Estratégia obrigatória na criação

Estratégia

Comportamento

OBJECTIVE_DRIVEN

Conversa livre e natural para alcançar objetivos cadastrados.

FLOW_DRIVEN

Execução estruturada por etapas, condições, ações e transições.

Separação de UX

Objective Builder e Flow Builder são telas e experiências distintas. Knowledge, Tools, Policies, Memory, Channels,

Model, Testing e Observability são capacidades compartilhadas.

3.2 Objective-driven

-

Objective: resultado desejado, como obter CNPJ, cidade, motivação, orçamento ou reunião.

-

Priority e dependencies.

-

Completion Condition estruturada.

-

Estados PENDING, IN_PROGRESS, COMPLETED, SKIPPED e FAILED.

-

Retry/attempt policy para evitar repetição artificial.

-

O agente pode responder perguntas do usuário e depois retomar naturalmente um objetivo pendente.

-

Valor extraído e evidência/contexto ficam persistidos no ObjectiveState.

3.3 Flow-driven

-

Flow versionado composto por FlowNodes e FlowTransitions.

•

Nodes: Message, Question, Action, Condition, Delay, Transfer, Objective Group e End.

•

FlowState mantém posição e contexto da execução.

•

Indicado para processos determinísticos, compliance e sequências obrigatórias.

4. Tools, integrações, knowledge e runtime

4.1 Tools & Capability Platform

Tool é uma capability independente do Agent. O Agent recebe acesso por ToolBinding e permissions, nunca por acoplamento direto.

•

Tool, ToolVersion, ToolBinding, ToolPermission, ExecutionPolicy e ToolExecution.

•

Tools podem ser Native, Integration-backed, API ou Custom.

•

Timeout, retries, idempotency, approvals e limits são parte da política de execução.

4.2 Integration Hub

•

Connector, Connection, Credential, WebhookEndpoint e scopes.

•

OAuth, API Key e autenticações customizadas.

•

Custom API connector para sistemas não suportados nativamente.

•

Health, reconnect e credential rotation.

4.3 Knowledge & Memory

•

KnowledgeBase e KnowledgeSource por escopo; ingestão, indexação e retrieval permission-aware.

•

Memory de sessão, memória de longo prazo quando permitida e políticas de retenção.

•

Knowledge e Memory são conceitos separados.

4.4 Agent Runtime

•

Resolve tenant/domain/deployment/version; carrega contexto, memory, knowledge e policies.

•

Executa Objective Engine ou Flow Engine.

•

Realiza model/tool calls autorizadas e persiste estado, traces, costs e audit.

- Suporta retries, idempotency, queueing, concurrency, fallback e human handoff.

5. Commercial Platform - WAVER → Revenda → Cliente

Modelo B2B2B

A WAVER administra a rede e o catálogo; a Revenda vende, contrata e opera sua carteira; o Cliente usa o Customer Workspace. O cliente final não acessa o Marketplace.

Portal	Responsabilidade
WAVER Control	Revendas, catálogo privado, produtos, pricing-base, contratos com revendas, billing, risco, auditoria e operações.
WAVER Partner	Clientes, propostas, contratos, catálogo autorizado, licenças, billing, dunning, suporte e renovações.
Customer Workspace	Domains, Agents, Tools liberadas, integrações, knowledge, channels e usage.

5.1 Marketplace privado

- Visível somente para WAVER e Revendas.
- WAVER publica Products, Tools, Connectors, Solution Packs, Channels, Capacity e Services.
- A WAVER controla availability por revenda.
- O cliente vê apenas recursos contratados/provisionados.
- Arquitetura preparada para Publishers terceiros, certificação e revenue share no futuro.

6. Contratos, Entitlements e Provisioning

Regra central

Catalog ≠ Contract ≠ Entitlement ≠ Usage. Catalog diz o que pode ser vendido; Contract diz o que foi acordado;

Entitlement diz o que está tecnicamente liberado; Usage diz o que foi consumido.

Marketplace Listing → Quote → Contract → Contract Item → Entitlement → Provisioning → Customer

Workspace

Entidade

Responsabilidade

Contract / ContractVersion

Acordo e histórico imutável de alterações.

ContractItem

Produto/capability efetivamente contratada.

Entitlement

Permissão técnica resultante do contrato.

EntitlementLimit

Quota, quantidade, período ou restrição.

ProvisioningJob

Aplica capabilities de forma idempotente.

UsageRecord

Medição para limite, analytics e cobrança.

•

Upgrade/downgrade gera aditivo/nova versão, não sobrescrita.

•

Effective dates permitem mudanças futuras.

•

Remoção/expiração não apaga dados automaticamente.

•

Agent Builder só oferece capabilities para as quais existam Entitlements válidos.

7. Billing, inadimplência e rotina comercial

7.1 Estados financeiros/operacionais

Estado

Comportamento

CURRENT

Operação normal.

PAST_DUE

Cobrança e alertas, sem corte imediato.

GRACE_PERIOD

Tolerância configurável.

RESTRICTED

Bloqueio seletivo de expansão/novos deployments.

SUSPENDED

Execuções/canais bloqueados conforme política; dados preservados.

TERMINATED

Contrato encerrado; retenção inicia.

REACTIVATING

Regularização e restauração controlada.

- Revenda só pode bloquear clientes da própria carteira.
- Suspensão exige motivo, responsável, timestamp e AuditEvent.
- Suspensão, termination, retention e purge são processos diferentes.
- Nunca apagar dados por inadimplência.
- Regularização pode reativar automaticamente ou por aprovação.

7.2 Rotinas

Ator

Rotina principal

WAVER

Monitorar revendas, billing, catálogo, risco, usage, incidentes, suporte e performance.

Revenda

Nova venda, contrato, provisioning, cobrança, dunning, suporte, upsell, renovação e cancelamento.

Cliente

Usar e administrar somente os recursos contratados.

8. Painel WAVER Control e WAVER Partner

8.1 WAVER Control

- Executive Dashboard; Reseller Management; Customer Oversight; Private Marketplace; Catalog Governance.
- Pricing & Commercial Rules; Contracts; Billing & Receivables; Usage & Metering.
- Risk & Compliance; Support & Operations; Audit Center.

8.2 WAVER Partner

- Dashboard da carteira; Customers; Quotes/Proposals; Contracts; Entitlements; Licenses.
- Billing; Dunning; Marketplace privado; Usage; Support; Renewals.

8.3 Customer Workspace

- Sem Marketplace e sem wholesale/margem da revenda.
- Pode administrar Agents, Domains, Knowledge, Integrations e capabilities contratadas.
- Solicitações de expansão direcionam à revenda.

9. Arquitetura obrigatória de microsserviços

AR-001 - Service and Layer Isolation

WAVER SHALL use a distributed microservice architecture. Business capabilities SHALL be separated by bounded

contexts. Each service SHALL own its domain model, persistence, migrations and public contracts. Services SHALL

NOT access another service's database or internal implementation. Cross-service interaction SHALL occur only

through versioned APIs or asynchronous events.

9.1 Planes

Plane

Serviços principais

Commercial

Reseller, Customer, Contract, Catalog, Marketplace, Entitlement, Billing.

Agent

Agent Builder, Objective Engine, Flow Engine, Agent Runtime, Memory, Evaluation.

Capability

Tool Registry, Tool Runtime, Connector, Knowledge, Channel.

Automation

Trigger, Workflow, Scheduler, Event.

Platform

Identity & Access, Tenant, Audit, Notification, Observability, Usage/Metering.

9.2 Regra de propriedade

- Cada microsserviço possui API, domínio, persistence, migrations, eventos, testes, deploy e observabilidade próprios.
- Nenhum serviço faz query direta no banco de outro.
-

Contratos são compartilhados; implementações internas não.

•

Evitar distributed monolith: banco compartilhado e imports cruzados são violações arquiteturais.

9.3 Comunicação

•

Síncrona para decisões imediatas, por APIs versionadas.

•

Assíncrona/event-driven para propagação de mudanças e processos desacoplados.

•

Exemplo: ContractActivated → Event Bus → Entitlement, Provisioning, Billing, Notification e Audit.

•

Eventos devem ser versionados, idempotentes e observáveis.

10. Isolamento interno de cada microsserviço

Cada serviço adota Clean/Hexagonal Architecture, mantendo as camadas isoladas.

Camada

Responsabilidade

Domain

Entities, Value Objects, Domain Services, Domain Events e repository interfaces.

Application

Use Cases, Commands, Queries, DTOs e orchestration.

Adapters

HTTP/API, event consumers/producers e persistence adapters.

Infrastructure

PostgreSQL, broker, cache e providers externos.

Dependency Rule

Dependências apontam para dentro: Infrastructure → Adapters → Application → Domain. O Domain não conhece

PostgreSQL, framework HTTP, broker, Redis, cloud ou SDK de fornecedor.

10.1 Control Plane vs Data Plane

•

Control Plane: Agents, Contracts, Tools, Entitlements, Configurations e Deployments.

•

Data Plane: Conversations, Agent Runs, Tool Executions, Messages, Objectives e Flows.

•

Carga de execução não deve derrubar administração comercial ou configuração.

11. Segurança, governança e requisitos

enterprise

•

Strict tenant isolation em consultas, jobs e eventos.

- RBAC e ABAC quando necessário; least privilege.
- Secrets em secret manager/vault.
- Audit log para ações comerciais, administrativas e técnicas.

- MFA/SSO preparados para contas administrativas.
- LGPD: minimização, retenção, exportação e exclusão.
- Backups, restore e disaster recovery.
- Rate limits, quotas e budget controls.
- Idempotência em billing, provisioning, tool calls e suspensão.
- Observability: logs, metrics, traces, custos e correlação por tenant.
- Approval workflows para ações de alto impacto.
- Soft delete + retention antes de purge.

12. Testing, deployment e observability

12.1 Testing & Evaluation

- Playground por AgentVersion; TestSuite/TestCase; cenários; expected outcomes.
- Assertions de tool calls e policies; regressão antes de production.
- Comparação entre versões/modelos; quality scores e human review.

12.2 Deployment

- Draft → Version → Environment → Deployment/Release.
- Editar Agent não altera production automaticamente.

-

Rollback; config/secrets por ambiente; possibilidade futura de canary rollout.

12.3 Métricas

-

Task/objective completion, handoff, latency, cost, tool errors e quality.

-

MRR/ARR, churn, expansion, inadimplência, attach rate e margem.

-

Provisioning success, support SLA e incident rate.

13. Entidades conceituais principais

Contexto

Entidades

Tenancy

Organization, OrganizationRelationship, Workspace, Domain, User, Membership, Role, Permission

Agent

Agent, AgentVersion, AgentStrategy, ModelConfig, BehaviorConfig, Deployment

Objectives

ObjectiveDefinition, ObjectiveState, CompletionCondition, ExtractedValue

Flows

Flow, FlowVersion, FlowNode, FlowTransition, FlowState, FlowVariable

Capabilities

Tool, ToolVersion, ToolBinding, ToolPermission, Connector, Connection, Credential

Knowledge/Memory

KnowledgeBase, KnowledgeSource, Document, RetrievalConfig, MemoryConfig, MemoryEntry

Runtime

AgentRun, ExecutionContext, Session, Conversation, Message, ToolExecution

Commercial

ResellerProfile, CustomerProfile, Catalog, Product, SKU, Listing, PriceBook

Contracts

Contract, ContractVersion, ContractItem, Amendment, Renewal, Cancellation

Entitlements

Entitlement, EntitlementLimit, FeatureFlag, ProvisioningJob

Finance

Invoice, InvoiceLine, Payment, Credit, Adjustment, DunningCase, UsageRecord

Governance

Policy, ApprovalPolicy, AuditEvent, Log, Trace, Metric, Quota, RateLimit

14. Decisões já fechadas

-

WAVER é plataforma de agentes, não somente SDR.

- Domínios empresariais são fronteiras reais.
- Agent Builder usa Objective-driven ou Flow-driven como estratégia estrutural.
- Objetivos e fluxos não são misturados como uma única experiência.
- Tools são capabilities independentes e reutilizáveis.
- Marketplace é privado para WAVER e Revendas; cliente final não acessa.
- Contrato gera Entitlements técnicos.
- Revenda administra a própria carteira e pode suspender cliente inadimplente dentro das regras.
- Dados não são apagados por inadimplência.
- Produto será construído em microsserviços isolados.
- Cada microsserviço possui banco/persistência e contratos próprios.
- Camadas internas também são isoladas via Clean/Hexagonal Architecture.
- Control Plane e Data Plane são separados conceitualmente.
- Interface principal é clara, técnica e sem expor parâmetros de LLM desnecessariamente.

15. Decisões ainda abertas antes do

desenvolvimento

Tema	Decisão necessária
Ownership	Regras formais de ownership e transferência de clientes entre revendas.
Billing	Quem cobra quem; gateway; fiscal; proration e currency.
Pricing	

Autonomia da revenda, margens e approvals.

Reseller default

Política quando a própria revenda fica inadimplente com a WAVER.

Contract engine

Assinatura dentro da plataforma ou registro externo inicialmente.

Entitlements

Granularidade: org/workspace/domain/agent/user/capability.

Marketplace

Publisher de terceiros e revenue share: quando ativar.

Support

L1/L2/L3, SLAs e impersonation.

Runtime stack

Broker/event bus, persistence e estratégia de long-running jobs.

Data isolation

Database-per-service obrigatório; decidir isolamento tenant físico/lógico por serviço.

Knowledge

Vector store, freshness, citations e permission-aware retrieval.

Channels

Primeiros canais de production e abstração de providers.

Compliance

Retenção, consentimento, data residency e requisitos enterprise.

Native Apps

Quais serão produtos estratégicos e quais apenas referência.

16. Ordem recomendada para iniciar

engenharia

•

1. Congelar terminologia e bounded contexts.

•

2. Produzir Master Domain Model e mapa definitivo de microsserviços.

•

3. Definir contratos síncronos e eventos assíncronos entre serviços.

•

4. Definir tenancy, IAM e autorização transversal.

•

5. Especificar Commercial Plane: Catalog → Contract → Entitlement → Provisioning.

•

6. Especificar Agent Plane: Builder → Version → Deployment → Runtime.

•

7. Especificar Objective Engine e Flow Engine como motores separados.

•

8. Definir Tool/Connector contracts e security model.

•

9. Definir observability, audit, usage e cost metering desde o início.

•

10. Produzir UMLs, state machines e sequence diagrams.

•

11. Desenhar WAVER Control, WAVER Partner e Customer Workspace.

•

12. Só então selecionar stack final e iniciar implementação por vertical slice.

Próximo marco

O próximo documento técnico deve ser o WAVER Microservices & Domain Architecture: lista definitiva de serviços,

ownership de dados, APIs, eventos, dependências permitidas, bancos, state machines e diagramas de sequência.

Apêndice A. Regras arquiteturais não

negociáveis

ID	Regra
AR-001	Toda capability de negócio pertence a um bounded context explícito.
AR-002	Nenhum microsserviço acessa o banco de outro serviço.
AR-003	Nenhum serviço importa implementação interna de outro.
AR-004	Comunicação cross-service somente por API versionada ou evento versionado.
AR-005	Cada serviço possui migrations, testes, deploy e observability próprios.
AR-006	Eventos e comandos com side effects devem ser idempotentes.
AR-007	Domain não depende de infraestrutura/framework.
AR-008	Tenant context deve ser propagado e validado em toda chamada/evento.
AR-009	Ações sensíveis exigem autorização e audit trail.
AR-010	Configuração editável nunca altera production sem version/deployment.
AR-011	Entitlements são a autoridade técnica de acesso a capabilities comerciais.
AR-012	Suspensão é reversível e distinta de termination/purge.
AR-013	Data Plane não depende da disponibilidade do painel administrativo para workloads já autorizados.
AR-014	Falhas cross-service usam timeout, retry controlado, circuit breaking/outbox quando aplicável.
AR-015	Contratos públicos e schemas possuem política de compatibilidade/versionamento.